

Dzień 16 - NAT/PAT, ACL, nftables i diagnostyka firewalla

Junior Network Engineer - szczegółowe notatki z wykonanego laboratorium

Cel dnia

Dzień 16 zamyka dział „Analiza ruchu i zapory”. Celem było zrozumienie i praktyczne przećwiczenie:

- NAT i PAT oraz mechanizmu MASQUERADE,
- różnicy między ruchem INPUT, OUTPUT i FORWARD,
- podstaw ACL / reguł firewalla,
- podstaw nftables oraz współpracy z istniejącymi regułami iptables-nft,
- stanów połączeń ESTABLISHED,RELATED,
- diagnozy kontrolowanej usterki według schematu: objaw → hipoteza → test → interpretacja → naprawa → weryfikacja,
- bezpiecznego rollbacku po laboratorium.

Uwaga: Ważne: wyniki opisane jako „wykonane” poniżej pochodzą z faktycznie wykonanej sesji. Nie są to wyniki przykładowe.

1. Topologia laboratorium

Utworzyliśmy izolowanego klienta Linux w network namespace:

```
klient16
10.16.0.2/24
|
| veth16-k
| veth16-r
v
Ubuntu – router/firewall
10.16.0.1/24
en01: 192.168.0.51/24
|
v
router domowy
192.168.0.1
|
v
Internet
```

Ubuntu pełniło jednocześnie rolę:

1. bramy dla klienta 10.16.0.2,
2. routera przekazującego pakiety,
3. urządzenia wykonującego NAT/MASQUERADE,
4. firewalla kontrolującego ruch FORWARD.

2. NAT — po co jest potrzebny?

Adres 10.16.0.2 jest adresem prywatnym. Router domowy nie powinien otrzymywać z naszego laboratorium pakietu, którego źródłem pozostaje 10.16.0.2, jeżeli nie zna trasy powrotnej do tej sieci.

Dlatego Ubuntu wykonuje translację:

```
PRZED NAT:
10.16.0.2 -> 1.1.1.1

PO MASQUERADE na eno1:
192.168.0.51 -> 1.1.1.1
```

W laboratorium użyliśmy reguły:

```
sudo nft add rule ip dzien16 postrouting \
ip saddr 10.16.0.0/24 oifname "eno1" masquerade
```

Znaczenie:

- ip saddr 10.16.0.0/24 — źródłem jest nasza sieć laboratoryjna,
- oifname "eno1" — pakiet wychodzi przez eno1,
- masquerade — źródłowy adres IP zostaje zastąpiony adresem interfejsu wychodzącego.

NAT a PAT

W uproszczeniu:

- **NAT** — translacja adresów,
- **PAT** — wiele połączeń/hostów może korzystać ze wspólnego adresu dzięki rozróżnianiu połączeń także portami,
- **DNAT / port forwarding** — zmieniany jest adres docelowy; typowy przykład to przekierowanie portu z routera do serwera w LAN.

Nie należy mylić NAT z:

- DHCP — przydziela konfigurację IP klientom,
- VLAN — logicznie rozdziela sieć,
- routingiem — wybiera drogę pakietu.

3. INPUT, OUTPUT I FORWARD

Najważniejsza reguła pamięciowa:

```
D0 routera/Ubuntu -> INPUT
Z routera/Ubuntu -> OUTPUT
PRZEZ routera/Ubuntu -> FORWARD
```

Przykłady:

Kluczowe pytanie brzmi: czy urządzenie jest celem pakietu, źródłem pakietu, czy tylko przekazuje go dalej?

4. Przygotowanie routingu klienta

Klient otrzymał adres:

```
10.16.0.2/24
```

Ubuntu po stronie laboratorium:

```
10.16.0.1/24
```

Dodaliśmy klientowi bramę:

```
sudo ip netns exec klient16 ip route add default via 10.16.0.1
```

Trasa klienta:

```
default via 10.16.0.1 dev veth16-k
10.16.0.0/24 dev veth16-k ... src 10.16.0.2
```

Na Ubuntu `net.ipv4.ip_forward = 1`, więc kernel mógł routować IPv4.

Sprawdzenie decyzji routingu:

```
ip route get 192.168.0.1 from 10.16.0.2 iif veth16-r
```

Rzeczywisty wynik:

```
192.168.0.1 from 10.16.0.2 dev eno1  
cache iif veth16-r
```

Interpretacja: pakiet przychodzący z veth16-r powinien wyjść przez eno1.

5. Pierwsza usterka — NAT jest, ale klient nadal nie ma łączności

Przed naprawą:

```
sudo ip netns exec klient16 ping -c 3 1.1.1.1
```

Rzeczywisty wynik:

```
3 packets transmitted, 0 received, 100% packet loss
```

Sprawdziliśmy istniejący firewall. Łańcuch FORWARD miał politykę drop i przechodził m.in. przez DOCKER-USER.

Nie modyfikowaliśmy bezpośrednio tabel zarządzanych przez Docker/Tailscale. Własny NAT trzymaliśmy w tabeli dzien16.

Diagnostyka krok po kroku

Objaw: klient ma adres i bramę, ale nie osiąga routera ani Internetu.

Hipoteza: routing jest poprawny, lecz firewall blokuje FORWARD.

Test 1 — routing:

```
ip route get 192.168.0.1 from 10.16.0.2 iif veth16-r
```

Wynik wskazał dev eno1 — routing był poprawny.

Test 2 — czy pakiet dociera od klienta do Ubuntu?

```
sudo tcpdump -ni veth16-r icmp
```

Rzeczywisty wynik zawierał trzy żądania:

```
10.16.0.2 > 192.168.0.1: ICMP echo request
```

Pakiety docierały do Ubuntu.

Test 3 — dopuszczenie ruchu wychodzącego w DOCKER-USER:

```
sudo iptables -I DOCKER-USER 1 \  
-i veth16-r -o eno1 -s 10.16.0.0/24 -j ACCEPT
```

Licznik później pokazał 3 pakiety trafiające w regułę, ale ping nadal nie wracał.

Test 4 — obserwacja eno1:

```
sudo tcpdump -ni eno1 icmp
```

Rzeczywisty wynik:

```
192.168.0.51 > 192.168.0.1: ICMP echo request  
192.168.0.1 > 192.168.0.51: ICMP echo reply
```

To był kluczowy dowód:

- pakiet wyszedł z Ubuntu,
- MASQUERADE działał (10.16.0.2 stał się 192.168.0.51),
- router 192.168.0.1 odpowiadał,
- problem dotyczył drogi powrotnej przez firewall.

6. ESTABLISHED,RELATED — ruch powrotny

Dodaliśmy regułę:

```
sudo iptables -I DOCKER-USER 2 \  
-i eno1 -o veth16-r \  
-d 10.16.0.0/24 \  
-m conntrack --ctstate ESTABLISHED,RELATED \  
-j ACCEPT
```

Znaczenie:

- odpowiedź przychodzi przez eno1,
- ma zostać przekazana do klienta przez veth16-r,
- dopuszczamy ruch należący do już zestawionego połączenia lub z nim powiązany.

Nie jest to reguła „wpuszczaj wszystko z Internetu”.

Po dodaniu reguły:

```
sudo ip netns exec klient16 ping -c 3 192.168.0.1
```

Rzeczywisty wynik:

```
3 packets transmitted, 3 received, 0% packet loss
```

Następnie:

```
sudo ip netns exec klient16 ping -c 3 1.1.1.1
```

Rzeczywisty wynik:

```
3 packets transmitted, 3 received, 0% packet loss  
rtt min/avg/max/mdev = 8.124/8.256/8.512/0.181 ms
```

Pełna ścieżka zaczęła działać.

7. ACL / firewall — kontrola kto może dokąd

ACL i reguły firewalla służą do określenia m.in.:

- źródła ruchu,
- celu,
- protokołu,
- portu,
- kierunku,
- decyzji: ACCEPT / DROP / REJECT.

Przykład biznesowy:

VLAN 10 PRACOWNICY	-> kamera	VLAN 50	ZEZWÓL
VLAN 20 GOŚCIE	-> kamera	VLAN 50	BLOKUJ
VLAN 20 GOŚCIE	-> Internet		ZEZWÓL

VLAN sam rozdziela domeny logiczne. Gdy istnieje routing między VLAN-ami, ACL/firewall może ograniczyć komunikację pomiędzy nimi.

8. Kontrolowana awaria — DROP dla 1.1.1.1

Celowo dodaliśmy:

```
sudo iptables -I DOCKER-USER 1 \  
-i veth16-r \  
-s 10.16.0.0/24 \  
-d 1.1.1.1 \  
-p icmp \  
-j DROP
```

Po zmianie:

```
sudo ip netns exec klient16 ping -c 3 1.1.1.1
```

Rzeczywisty wynik:

```
3 packets transmitted, 0 received, 100% packet loss
```

Sprawdzenie:

```
sudo iptables -L DOCKER-USER -v -n --line-numbers
```

Rzeczywisty licznik reguły:

```
3 252 DROP ... 10.16.0.0/24 -> 1.1.1.1
```

Dokładnie trzy wysłane pingi trafiły w regułę DROP.

Naprawa

```
sudo iptables -D DOCKER-USER 1
```

Weryfikacja

```
sudo ip netns exec klient16 ping -c 3 1.1.1.1
```

Rzeczywisty wynik po naprawie:

```
3 packets transmitted, 3 received, 0% packet loss  
rtt min/avg/max/mdev = 8.113/8.172/8.216/0.043 ms
```

9. Procedura „co zgłasza klient?”

Zgłoszenie: „Internet nie działa”

1. ****Objaw**** — ustal, czy nie działa wszystko, czy tylko konkretna usługa.
2. ****Adres klienta**** — ip addr, ipconfig.
3. ****Trasa/brama**** — ip route, ip route get ..., route print.
4. ****Łączność do bramy**** — ping .
5. ****Routing na routerze**** — sprawdź tablicę routingu i ip_forward.
6. ****Firewall/ACL**** — iptables -L ... -v -n --line-numbers, nft list ruleset.
7. ****Obserwacja pakietu przed i po routerze**** — tcpdump -ni
8. ****NAT**** — sprawdź, czy na interfejsie zewnętrznym źródło jest tłumaczone.
9. ****Napraw tylko potwierdzoną przyczynę.****
10. ****Powtórz dokładnie ten sam test****, który wcześniej nie działał.

Przykład z dzisiejszego laboratorium

Objaw:
klient16 nie może pingować 1.1.1.1.

Hipoteza:
firewall blokuje ruch.

Test:
liczniki DOCKER-USER + tcpdump na veth16-r i eno1.

Interpretacja:
request wychodzi po NAT, router odpowiada, odpowiedź nie wraca do klienta.

Naprawa:
dopuszczenie ruchu ESTABLISHED,RELATED.

Weryfikacja:
3/3 odpowiedzi, 0% strat.

10. nftables i iptables-nft — ważna uwaga z naszego serwera

Na serwerze istnieją reguły związane z Dockerem i Tailscale. Widzieliśmy również ostrzeżenie, że część tabel jest zarządzana przez iptables-nft.

Dlatego w laboratorium:

- nie wykonywaliśmy globalnego flush ruleset,
- nie czyściliśmy istniejących tabel,
- NAT laboratoryjny utworzyliśmy w osobnej tabeli ip dzien16,
- reguły użytkownika dla ścieżki Dockera dodaliśmy do DOCKER-USER,
- na końcu wykonaliśmy rollback.

To jest ważna praktyka: najpierw ustal, kto zarządza firewallem, dopiero potem zmieniaj reguły.

11. Rollback wykonany po laboratorium

Usunęliśmy dwie laboratoryjne reguły z DOCKER-USER i zweryfikowaliśmy, że łańcuch wrócił do pustego stanu.

Następnie:

```
sudo nft delete table ip dzien16
```

Weryfikacja:

```
sudo nft list table ip dzien16
```

Rzeczywisty wynik:

```
Error: No such file or directory
```

W tym przypadku błąd był oczekiwanym potwierdzeniem, że tabela już nie istnieje.

Na końcu:

```
sudo ip netns del klient16
ip netns list
```

ip netns list zwróciło pusty wynik — namespace został usunięty.

12. Ściąga poleceń

```
# routing
ip route
ip route get 192.168.0.1 from 10.16.0.2 iif veth16-r
sysctl net.ipv4.ip_forward

# nftables
sudo nft list ruleset
sudo nft list table ip dzien16
sudo nft -a list chain ip dzien16 forward

# iptables / DOCKER-USER
sudo iptables -L DOCKER-USER -v -n --line-numbers

# obserwacja ruchu
sudo tcpdump -ni veth16-r icmp
sudo tcpdump -ni eno1 icmp

# test z namespace
sudo ip netns exec klient16 ping -c 3 192.168.0.1
sudo ip netns exec klient16 ping -c 3 1.1.1.1
```

13. Co trzeba zapamiętać na Junior Network Engineer

1. NAT nie zastępuje routingu ani firewalla.
2. MASQUERADE zmienia źródło na adres interfejsu wychodzącego.
3. INPUT = ruch do routera/hosta.
4. OUTPUT = ruch utworzony przez router/host.
5. FORWARD = ruch przekazywany przez router.
6. ESTABLISHED,RELATED jest typowym sposobem dopuszczenia ruchu powrotnego.
7. Kolejność reguł ma znaczenie — wcześniejszy DROP może zatrzymać pakiet przed późniejszym ACCEPT.
8. Liczniki pakietów są dowodem, czy reguła rzeczywiście pasuje do ruchu.
9. tcpdump po obu stronach routera pomaga ustalić, gdzie znika pakiet.
10. Po zmianie zawsze wykonuj test, a po laboratorium rollback i jego weryfikację.

14. Postęp po Dniu 16

Wykonane i zweryfikowane:

- izolowany klient klient16,
- routing klienta przez Ubuntu,
- NAT/MASQUERADE,
- diagnostyka FORWARD,
- dopuszczenie ruchu wychodzącego i powrotnego,
- ESTABLISHED,RELATED,
- dostęp klienta do routera i Internetu,
- kontrolowana reguła DROP,
- diagnostyka na licznikach i tcpdump,
- naprawa i ponowna weryfikacja,
- pełny rollback laboratorium.

Trudność do dalszej powtórki: podczas lekcji INPUT, OUTPUT i FORWARD początkowo się mieszały. Końcowa seria została rozwiązana poprawnie: INPUT, OUTPUT, FORWARD.

Następny krok kursu: Dzień 17 — RouterOS/CHR/WinBox, interfejsy, bridge, Safe Mode, backup konfiguracji i export.